

CRUD Local

→ Objetivo: Simular um mini Crud no front end, buscando entender como funciona Update e o Delete em formulário no front end

📁 Arquivo: `app.ts`

Caminho: `Projeto05_CRUDLocal/src/app/app.ts`



```
1 import { Component, signal } from '@angular/core';
2
3 @Component({
4   selector: 'app-root',
5   standalone: true,
6   templateUrl: './app.html',
7 })
8 export class App {
9
10   // Simulação do Banco de Dados em Memória (Lista de produtos)
11   listaProdutos = signal(['Notebook', 'Mouse', 'Teclado'])
12
13   adicionar(novo : string) {
14     if(novo) {
15       // Adiciona o novo item no final da lista
16       this.listaProdutos.update(atual => [...atual, novo]);
17     }
18   }
19
20   remover(itemNome : string) {
21     // Remove o item da lista
22     this.listaProdutos.update(atual => atual.filter(p => p !== itemNome));
23   }
24 }
```

● Lógica de Adição: Imutabilidade com Spread

Na função `adicionar(novo: string)`, o código utiliza o método `this.listaProdutos.update()`. A expressão `[...atual, novo]` é a chave da **imutabilidade**.

- **O que faz:** O operador `...` (spread) copia todos os itens do array `atual` para um novo espaço de memória e insere o novo item no final.

- **Por que assim:** Não alteramos a lista existente. Criamos uma "versão 2.0" da lista.

● Lógica de Remoção: Filtragem Imutável

Na função `remover(itemNome: string)`, utilizamos o método `.filter()`.

- **O que faz:** Ele percorre o array e gera um **novo array** contendo apenas os produtos que têm o nome diferente do que foi clicado (`p !== itemNome`).
 - **Engenharia:** O item não é "deletado" da lista antiga; ele simplesmente não é incluído na lista nova que o Signal passará a gerenciar.
-

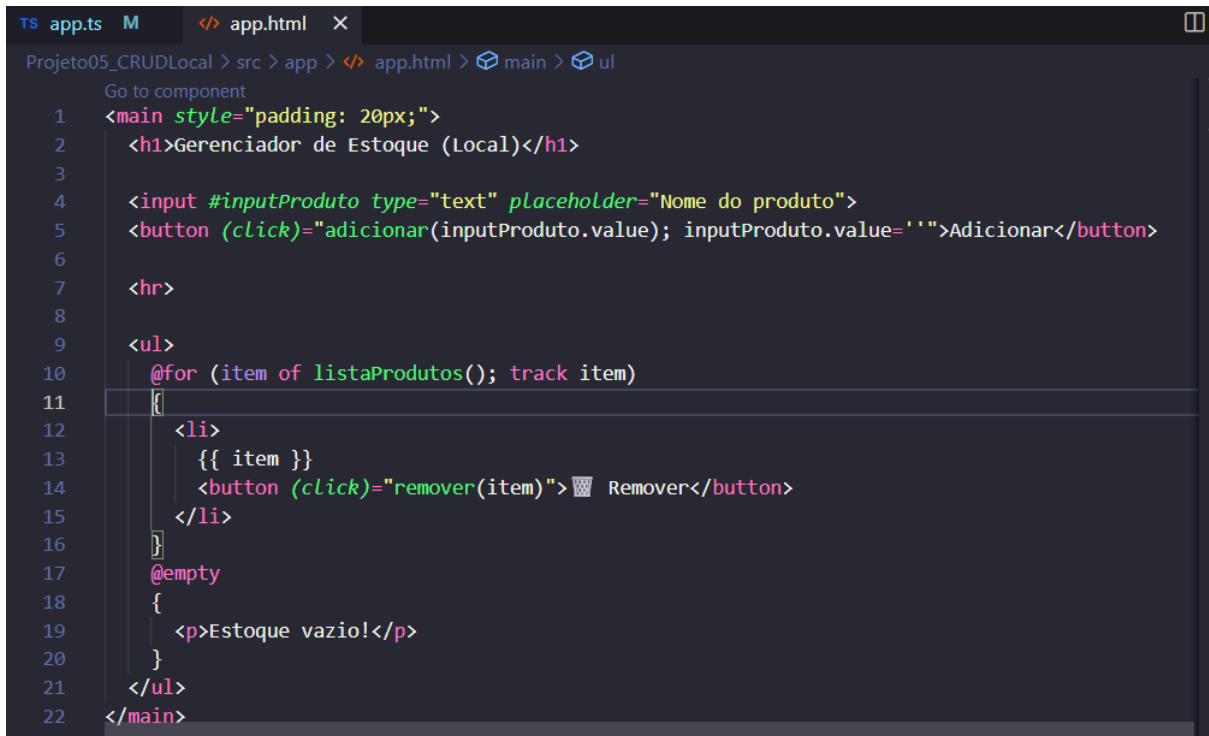
🧠 O Porquê das Listas Imutáveis (Performance e Memória)

Trabalhar com imutabilidade é o padrão ouro no Angular por três motivos técnicos fundamentais:

1. **Comparação por Referência (Endereço de Memória):** O Angular e os Signals não olham o conteúdo da lista item por item para saber se algo mudou, pois isso seria lento. Eles olham apenas o **endereço de memória**.
 - Se você usa `push()`, o endereço é o mesmo, e o Angular pode "achar" que nada mudou.
 - Ao criar uma lista nova, o endereço muda. O Angular percebe essa troca de "ponteiro" instantaneamente e atualiza a tela com precisão.
2. **Velocidade de Processamento:** Comparar se um endereço de memória `A` é diferente de `B` é uma das operações mais rápidas que existem no processador. Isso mantém a aplicação fluida, mesmo com milhares de dados.
3. **Segurança de Estado:** Como você nunca altera o dado original diretamente (mutação), você evita bugs onde um componente altera uma informação que outro componente também está usando sem aviso prévio. O estado do sistema evolui de forma previsível.

Arquivo: app.html

Caminho: Projeto05_CRUDLocal/src/app/app.html



```
1 <main style="padding: 20px;">
2   <h1>Gerenciador de Estoque (Local)</h1>
3
4   <input #inputProduto type="text" placeholder="Nome do produto">
5   <button (click)="adicionar(inputProduto.value); inputProduto.value=''>Adicionar</button>
6
7   <hr>
8
9   <ul>
10    @for (item of listaProdutos(); track item)
11    {
12      <li>
13        {{ item }}
14        <button (click)="remover(item)"> Remover</button>
15      </li>
16    }
17    @empty
18    {
19      <p>Estoque vazio!</p>
20    }
21  </ul>
22 </main>
```

A Variável de Template: #inputProduto

No Angular, o uso do símbolo # (como em #inputProduto) cria o que chamamos de **Variável de Referência de Template**.

- **O que ela é:** Ela funciona como um "ponteiro" ou uma instância direta da tag HTML no seu código. Ao declarar #inputProduto dentro da tag <input>, você ganha acesso a todo o objeto daquele campo sem precisar criar variáveis extras no TypeScript.
 - **Acesso aos Dados:** Quando usamos inputProduto.value, o Angular captura o texto exato que o usuário digitou naquele momento.
 - **Comportamento Padrão:** Por ser um campo de texto novo, seu valor inicial é uma string vazia (""), e ele se comporta como uma variável local que só existe dentro deste arquivo HTML.
-

Fluxo de Dados: Do HTML para o TypeScript

A integração entre a interface e a lógica segue caminhos bem definidos através dos eventos:

1. O Caminho da Adição

No botão "Adicionar", ocorre o seguinte fluxo:

1. **Captura:** O clique aciona a função `adicionar()`.
2. **Passagem de Parâmetro:** O valor contido em `inputProduto.value` é enviado como argumento para o TypeScript.
3. **Limpeza:** Logo após enviar o dado, o comando `inputProduto.value = ''` limpa o campo na tela, preparando-o para a próxima digitação.

2. O Caminho da Remoção e Listagem

Abaixo do formulário, o Angular utiliza o fluxo de iteração:

- **Interpolação `{{ item }}`:** O HTML recebe o dado do array `listaProdutos()` e o exibe para o usuário.
- **Event Binding (`click`):** No botão de remover, passamos o `item` atual da lista de volta para a função `remover()` no TypeScript, fechando o ciclo de interação.